National College of Ireland

Cloud Platform Programming

Cloud-based RESTful API

Outline

- Application Programming Interfaces
- Introduction to Amazon API Gateway

Application Programming Interfaces (APIs)

- Application Programming Interfaces (APIs)
  - Set of defined rules/specification that explain how computers or applications communicate with each other
  - Abstract the implementation details
  - Provide a “contract” defined in terms of actions, requests and responses

Application Programming Interfaces (APIs)

- APIs used for web applications
  - RESTful APIs
    - Request-response communication model between clients and servers
    - HTTP requests (HTTP verbs e.g. GET, POST, PUT, PATCH, DELETE)
  - WebSocket APIs
    - Bidirectional (two-way) communication between clients and servers

Amazon API Gateway

- A fully managed service that provides API proxy
- Enables developers to create, publish, maintain, monitor, and secure APIs at any scale
- Supports to create
  - RESTful APIs
  - WebSocket APIs
- Applications can access data, business logic, or functionality via the APIs

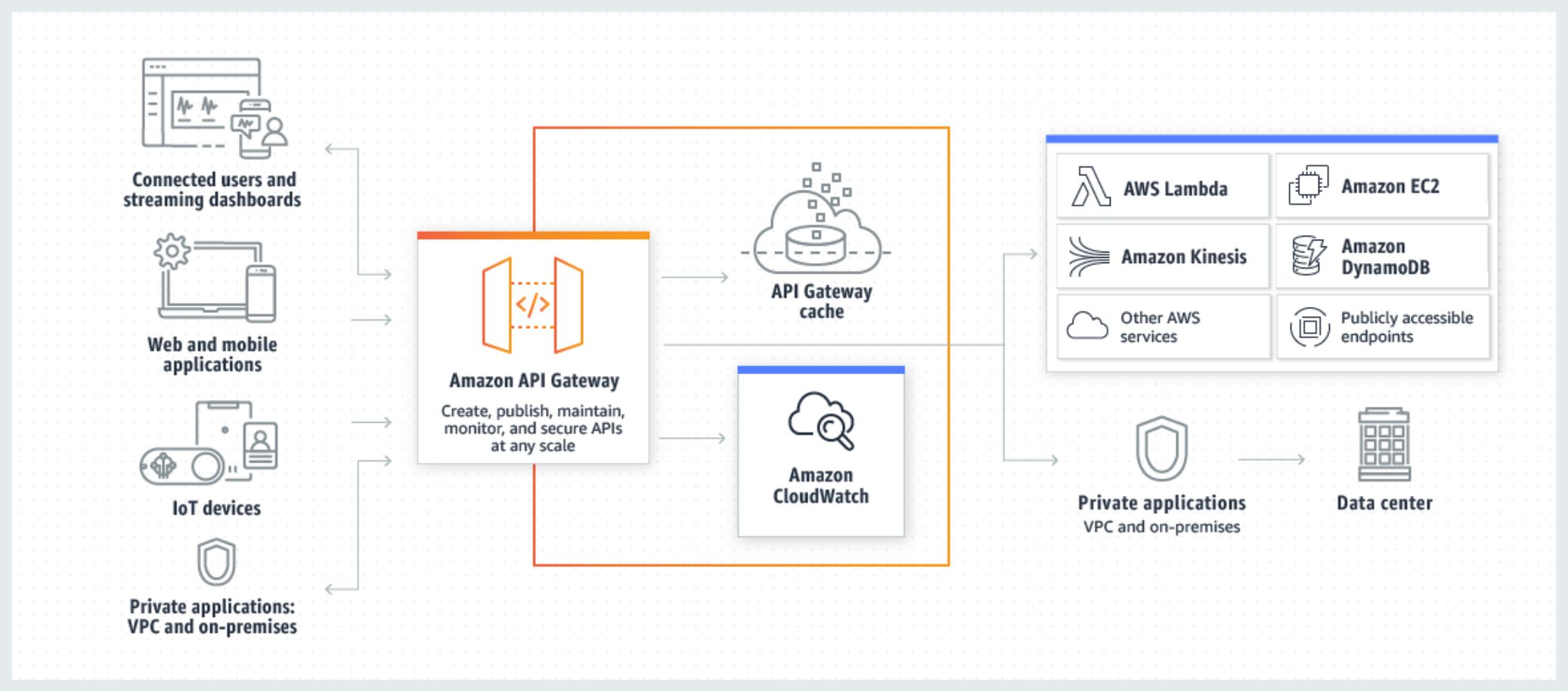
source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway

- Handles all the tasks pertaining to
  - Accepting and processing up to hundreds of thousands of concurrent API calls
  - Traffic management
  - Authorization and access control
  - Monitoring
  - API version management

source: https://aws.amazon.com/api-gateway/ .

Architecture of Amazon API Gateway



source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway – Benefits

- Efficient API development
- Support for RESTful APIs and websocket APIs
- API lifecycle management
- SDK generation
- API operations monitoring

source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway

- HTTP API
  - Build low-latency and cost-effective REST APIs
  - Works with: Lambda, HTTP backends

source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway

- HTTP API
  - Build low-latency and cost-effective REST APIs
  - Works with: Lambda, HTTP backends
- REST API
  - Develop a REST API where we have complete control over the request and response, and API management
  - Works with: Lambda, HTTP, AWS Services

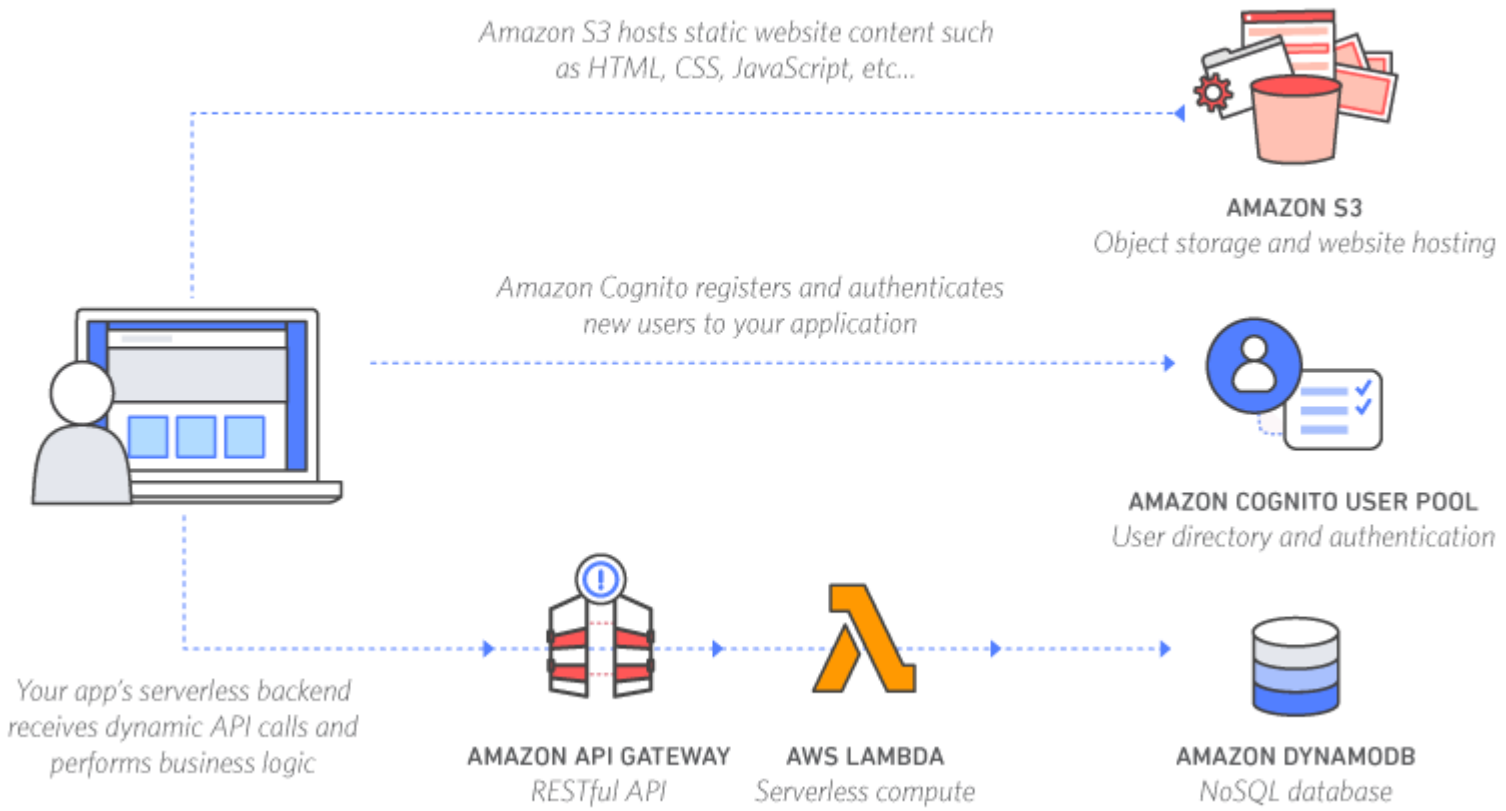
source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway

- HTTP API
  - Build low-latency and cost-effective REST APIs
  - Works with: Lambda, HTTP backends
- REST API
  - Develop a REST API where we have complete control over the request and response, and API management
  - Works with: Lambda, HTTP, AWS Services
- WebSocket API
  - Build WebSocket API using persistent connections for real-time use cases
    - e.g. chat applications, dashboards
  - Works with: Lambda, HTTP, AWS Services

source: https://aws.amazon.com/api-gateway/ .

Amazon API Gateway & AWS Lambda – An Example



source: https://aws.amazon.com/lambda

API Life Cycle

- Design and create an API
- Control access to the API
- Test the API
- Deploy the API
- Invoke the API
- Monitor the API execution

Creating APIs

- AWS Management Console – Amazon API Gateway Console
- AWS CLI
- Import an OpenAPI definition

RESTful API

- REST (Representational State Transfer) – an architectural style
- In REST any information/entity is considered a *resource*
- Uses HTTP verbs
  - e.g. GET, POST, PUT, PATCH, DELETE

Why REST?

- A client references a web resource using an URL
- A **representation** of the data is returned (e.g. nnn.html)
- The representation places the client in a new **state**
- When client selects a link on that HTML page
  - It accesses another resource
  - Client has moved to another state
  - Thus the client application **transfers** state with each resource representation

HTTP GET

- Retrieving information
  - e.g. A user types in www.amazon.com
- The browser creates a HTTP Header with no payload
- The HTTP Header identifies
  - The required action: GET (i.e. “retrieve a resource”)
  - The target (amazon.com)
    - Goes to target (amazon) and gets the resource

HTTP GET

- Retrieving information
  - e.g. A user types in www.amazon.com
- The browser creates a HTTP Header with no payload
- The HTTP Header identifies
  - The required action: GET (i.e. “retrieve a resource”)
  - The target (amazon.com)
    - Goes to target (amazon) and gets the resource

GET / HTTP / 1.1

Host: http://www.amazon.com

HTTP GET

- Retrieving information
  - e.g. A user types in www.amazon.com
- The browser creates a HTTP Header with no payload
- The HTTP Header identifies
  - The required action: GET (i.e. “retrieve a resource”)
  - The target (amazon.com)
    - Goes to target (amazon) and gets the resource

GET / HTTP / 1.1
Host: http://www.amazon.com



HTTP POST

- Updating information
- Unlike GET, has a payload (data)
  - e.g. The users fills out an online form
    - Book: “Python Programming”
    - Credit Card: Visa,...
- The browser creates a Header with a payload
- The HTTP Header identifies
  - The required action: POST (i.e. “update with this information”)
  - The target (amazon.com)
  - The payload – the data that has to be POSTed from the form data

HTTP POST

- Updating information
- Unlike GET, has a payload (data)
  - e.g. The users fills out an online form
    - Book: “Python Programming”
    - Credit Card: Visa,...
- The browser creates a Header with a payload
- The HTTP Header identifies
  - The required action: POST (i.e. “update with this information”)
  - The target (amazon.com)
  - The payload – the data that has to be POSTed from the form data

POST / HTTP / 1.1

Host: http://www.amazon.com

Header & Payload

- The header is the HTTP information
  - Request Type
  - Host
- Payload
  - The information you are sending
  - e.g. Book title and credit card information

Creating a RESTful API

- A REST API is a group of *resources* and *methods*
- In Amazon API Gateway, we build a REST API as a collection of resources and methods
- A *resource* is a logical entity that an application can access through a resource path
- A method corresponds to a REST API request that is submitted by the user of your API and the response returned to the user
  - an HTTP operation and a resource path
  - e.g. GET /books

API Endpoint Types

- An *edge-optimized API endpoint* is best for geographically distributed clients
  - API requests are routed to the nearest CloudFront Point of Presence (POP)
  - This is the default endpoint type for API Gateway REST APIs
  - Not available for HTTP APIs, WebSocket APIs
- A *regional API endpoint* is intended for clients in the same region
- A *private API endpoint* is an API endpoint that can only be accessed from your Amazon Virtual Private Cloud (VPC)
  - Available only for REST APIs

source: https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-endpoint-types.html/ .

Practical Task

- Create and test a simple REST API via the Amazon API Gateway Console
  - Create an API
  - Test an API
  - Deploy an API
  - https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-create-api-from-example.html

Resources

- Amazon API Gateway https://aws.amazon.com/api-gateway/
- Amazon API Gateway – Developer Guide https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html